

“Desarrollo de Plataforma Web para Gestión de Catálogo de la Librería ‘Jesús Mi Amigo’”

Arquitectura Orientada al Servicio

Doncente:Dr. Edwin Palomino Iriarte

Integrantes:

- | | |
|--------------------------|-----------|
| • MARQUEZ HIDALGO SAIR | U21217511 |
| • CARBAJAL SOLIS YHEISON | U20229758 |
| • REYES SALAZAR JOHN | U21319856 |
| • TERRAZAS RAMOS RICARDO | U21300765 |
| • PEREA CARRILLO JHORDY | U21217056 |





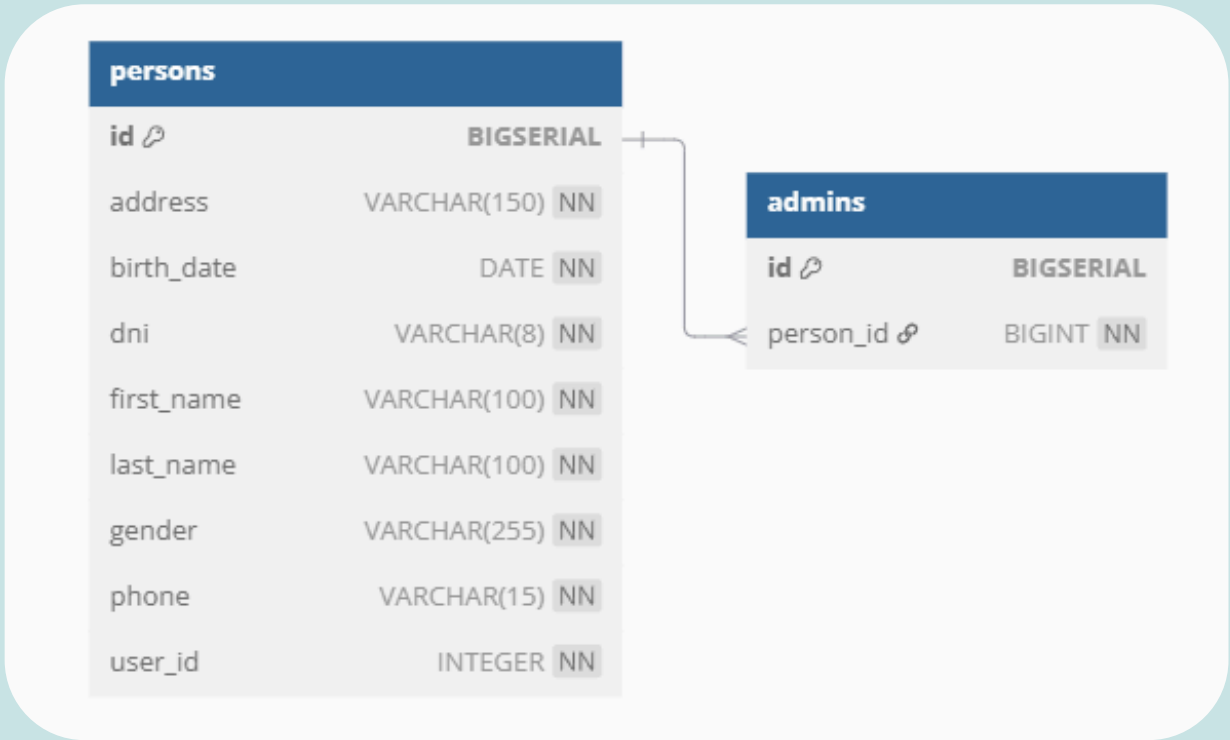
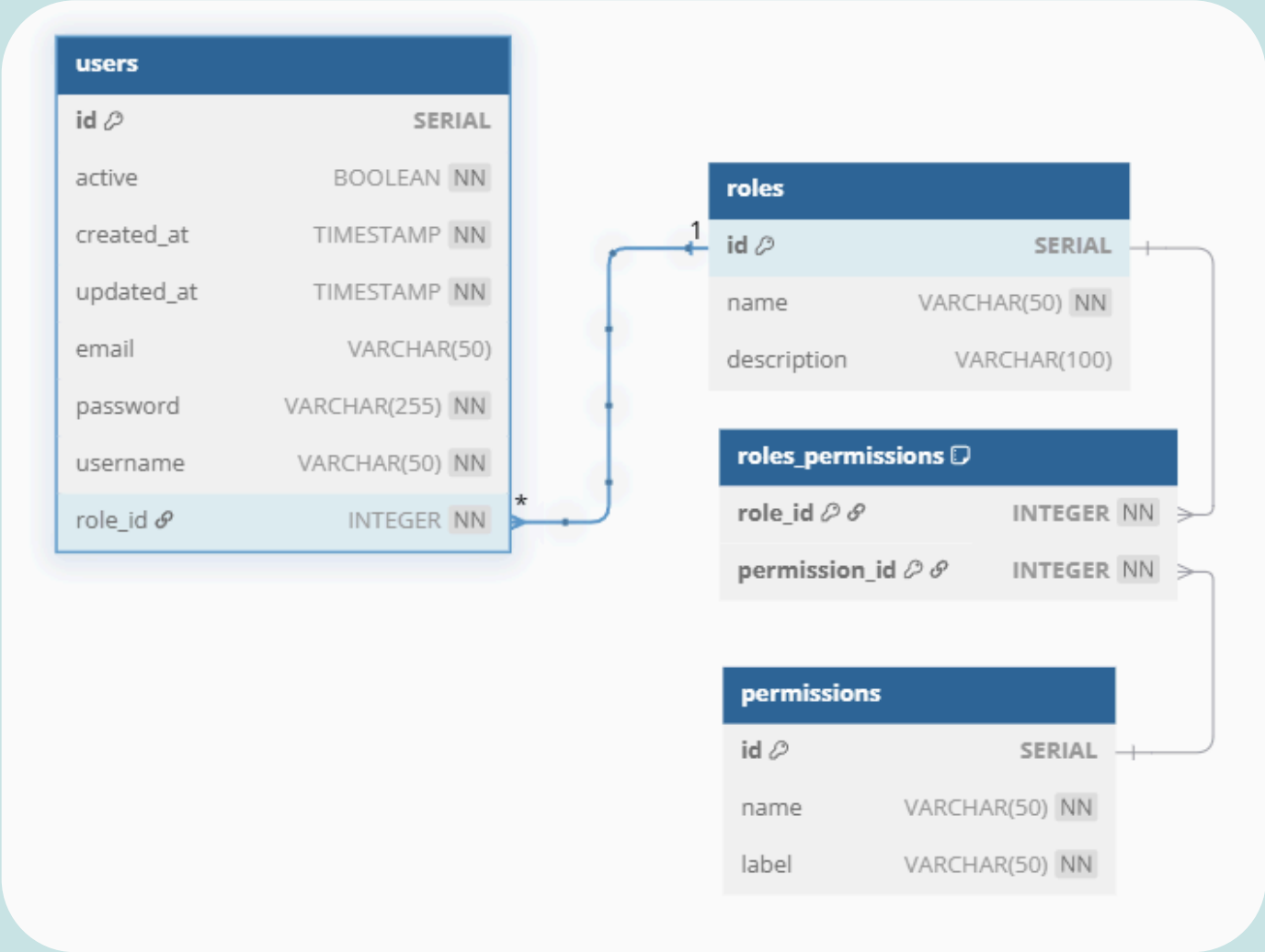
Introduccion

- Actualmente, la Librería 'Jesús Mi Amigo' gestiona su catálogo de forma manual, lo que genera duplicidad de registros, inventarios desactualizados y demoras en reposición.

Objetivo

- Nuestro objetivo es implementar una plataforma web con React y una API REST desacoplada, utilizando Arquitectura Orientada a Servicios (SOA) y Postgres, para automatizar la gestión de productos, stock, ventas, roles y permisos.

Base de datos Microservicios



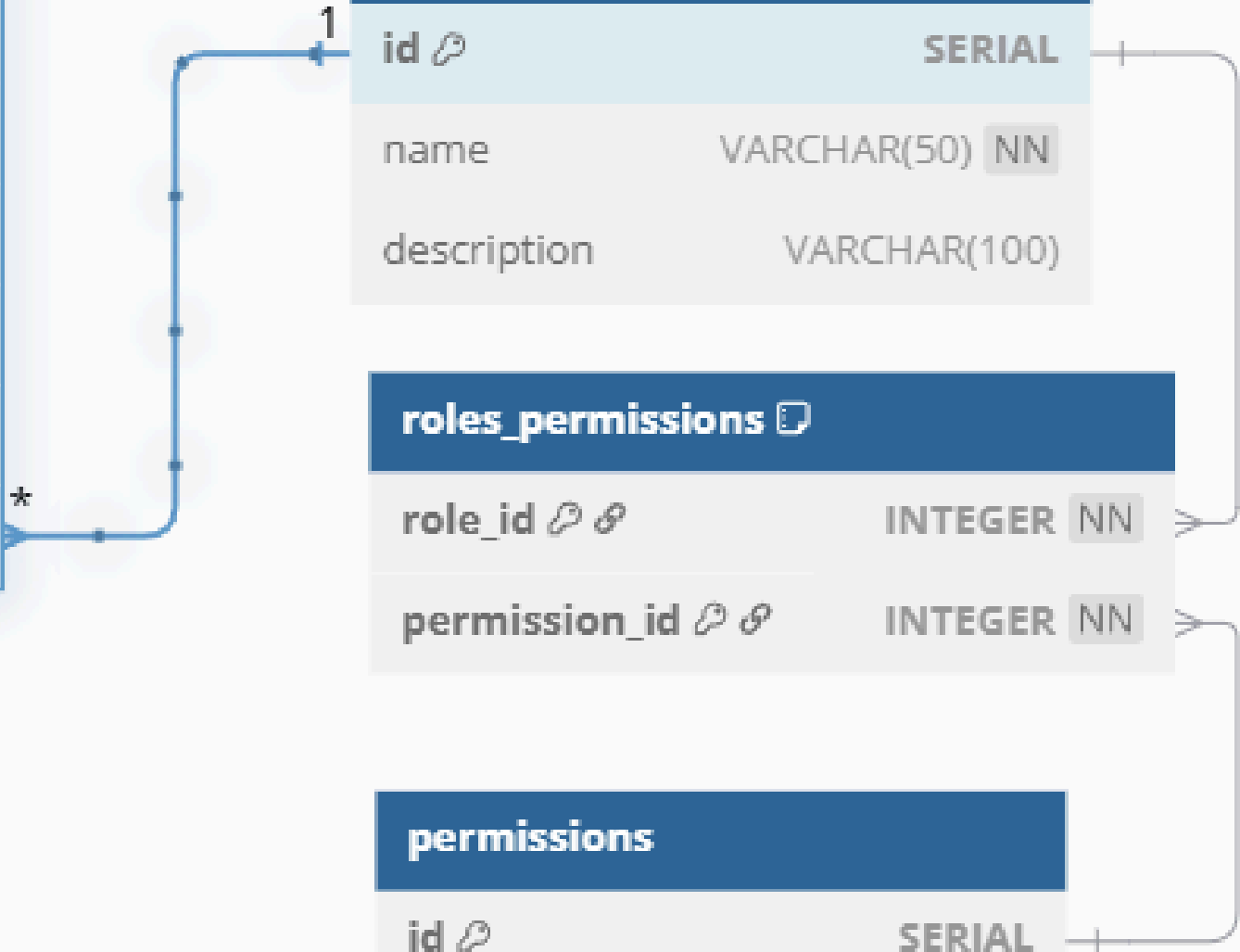
Base de datos > Microservicio Auth (Credenciales)

users		
id		SERIAL
active		BOOLEAN NN
created_at		TIMESTAMP NN
updated_at		TIMESTAMP NN
email		VARCHAR(50)
password		VARCHAR(255) NN
username		VARCHAR(50) NN
role_id		INTEGER NN

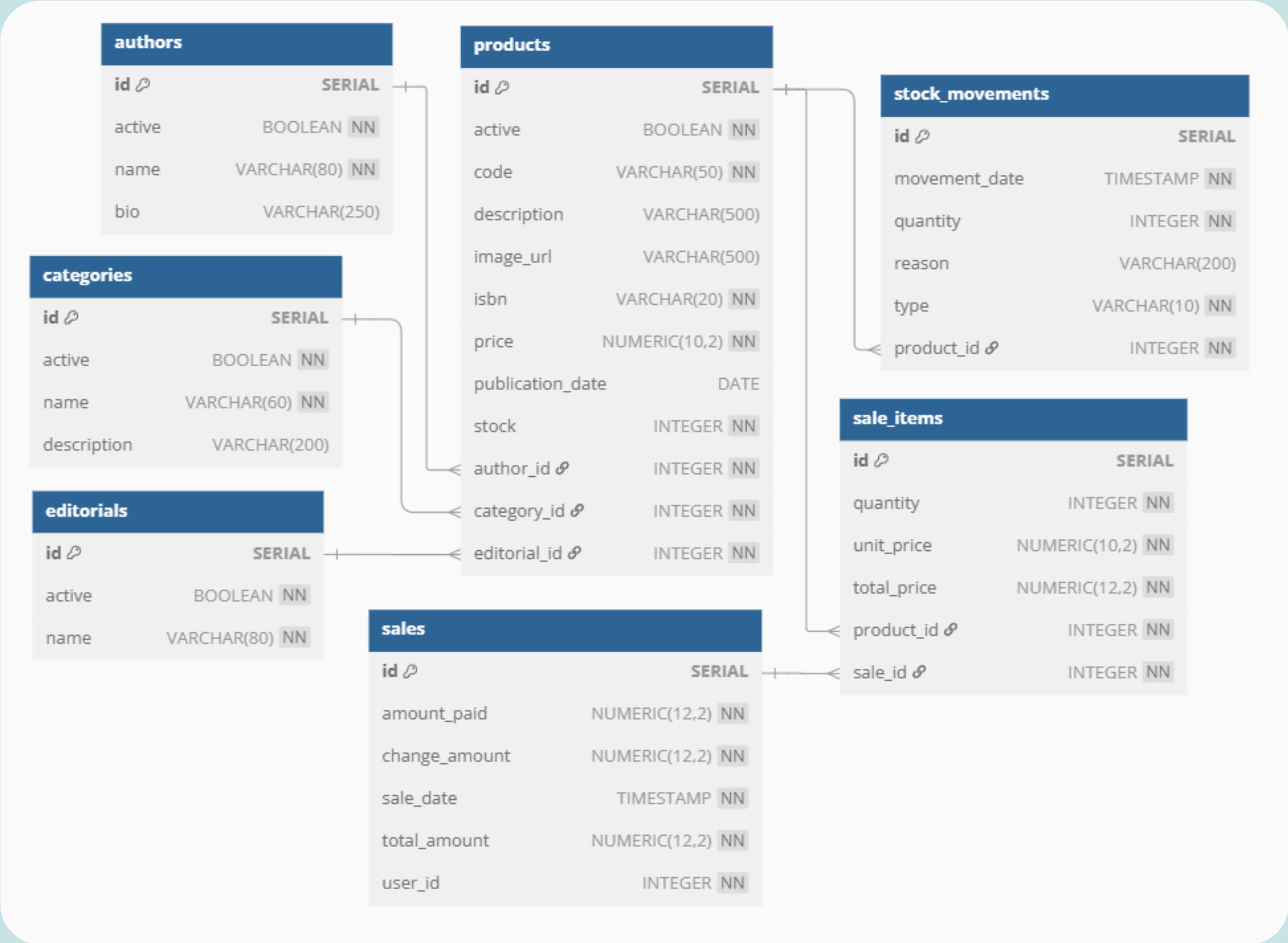
roles		
id		SERIAL
name		VARCHAR(50) NN
description		VARCHAR(100)

roles_permissions		
role_id		INTEGER NN
permission_id		INTEGER NN

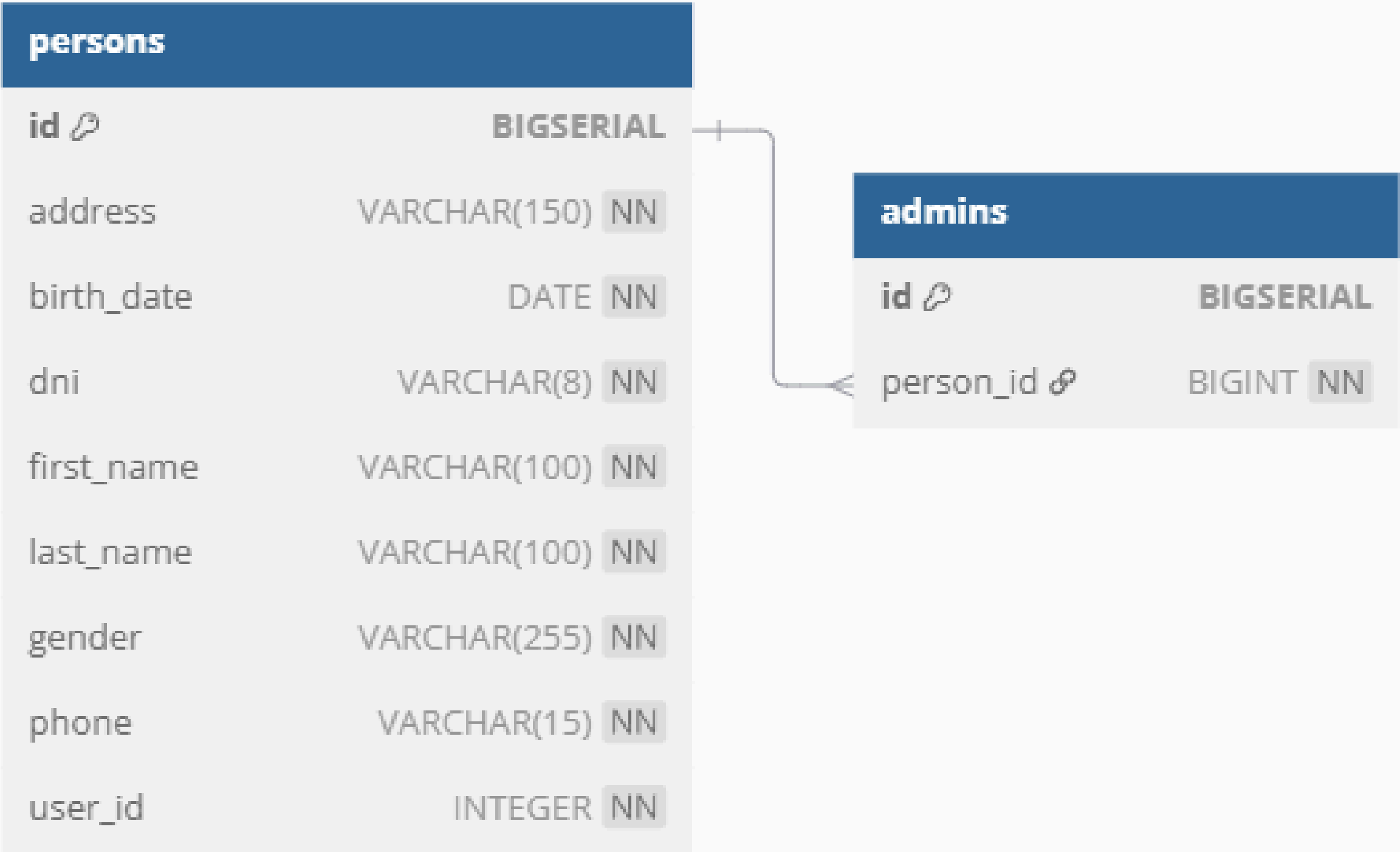
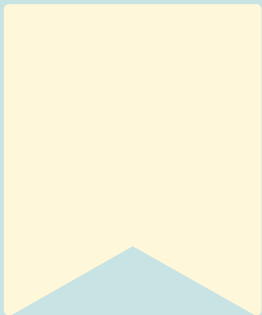
permissions		
id		SERIAL
name		VARCHAR(50) NN
label		VARCHAR(50) NN



Base de datos > Microservicio Productos



Base de datos > Microservicio Usuarios (Data de personas)



DTO con validaciones

© UserCreateDTO.java ×

```
1 package pe.jesusamigo.micro_users.user.dto;
2
3 import io.swagger.v3.oas.annotations.media.Schema;
4 import jakarta.validation.constraints.Email;
5 import jakarta.validation.constraints.NotBlank;
6 import jakarta.validation.constraints.Size;
7 import lombok.Data;
8
9 @Data 9 usages
10 public class UserCreateDTO {
11
12     @Schema(description = "Nombre de usuario único", example = "docente123", required = true)
13     @NotBlank(message = "El nombre de usuario no puede estar vacío.")
14     @Size(max = 50, message = "El nombre de usuario debe tener como máximo 50 caracteres.")
15     private String username;
16
17     @Schema(description = "Correo electrónico del usuario", example = "usuario@ejemplo.com", required = true)
18     @NotBlank(message = "El correo electrónico no puede estar vacío.")
19     @Email(message = "El correo electrónico debe tener un formato válido.")
20     @Size(max = 100, message = "El correo electrónico debe tener como máximo 100 caracteres.")
21     private String email;
22
23     @Schema(description = "Contraseña del usuario", example = "Segura123!", required = true)
24     @NotBlank(message = "La contraseña no puede estar vacía.")
25     @Size(min = 8, max = 255, message = "La contraseña debe tener entre 8 y 255 caracteres.")
26     private String password;
27
28     @Schema(description = "Nombre del rol asignado al usuario", example = "CAJERO", required = true)
29     @NotBlank(message = "El rol no puede estar vacío.")
30     private String role;
31 }
```


Servidor Eureka



System Status

Environment	test	Current time	2025-06-27T19:57:32 -0500
Data center	default	Uptime	00:05
		Lease expiration enabled	true
		Renews threshold	8
		Renews (last min)	16

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

[localhost](#)

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - host.docker.internal:api-gateway:8080
MICRO-AUTH	n/a (1)	(1)	UP (1) - host.docker.internal:micro-auth:8700
MICRO-PRODUCTS	n/a (1)	(1)	UP (1) - host.docker.internal:micro-products:8092
MICRO-USERS	n/a (1)	(1)	UP (1) - host.docker.internal:micro-users:8091

General Info

API Gateway

application.yml

```
1  spring:
2    application:
3      name: api-gateway
4    autoconfigure:
5      exclude:
6        - org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
7    cloud:
8      gateway:
9        default-filters:
10         - DedupeResponseHeader=Access-Control-Allow-Origin Access-Control-Allow-Credentials
11        routes:
12          # MICRO-AUTH
13          - id: micro-auth
14            uri: lb://micro-auth
15            predicates:
16              - Path=/api/auth/**
17          # MICRO-USERS (maneja usuarios, roles, permisos)
18          - id: micro-users-users
19            uri: lb://micro-users
20            predicates:
21              - Path=/api/users/**
22              - Path=/api/roles/**
23              - Path=/api/roles/*/permissions/**
24              - Path=/api/permissions/**
```

API Gateway

```
25      # MICRO-PRODUCTS (maneja inventario, productos, reportes, ventas)
26      - id: micro-products
27        uri: lb://micro-products
28        predicates:
29          - Path=/api/products/**
30          - Path=/api/authors/**
31          - Path=/api/categories/**
32          - Path=/api/editorials/**
33          - Path=/api/inventory/**
34          - Path=/api/stock-movements/**
35          - Path=/api/reports/**
36          - Path=/api/sales/**
37    server:
38      port: 8080
39    eureka:
40      client:
41        service-url:
42          defaultZone: http://localhost:8761/eureka
```

Comunicación entre Microservicios

© UserClient.java x

```
9
10 @Service 4 usages
11 @RequiredArgsConstructor
12 public class UserClient {
13
14     private final RestTemplate restTemplate;
15
16     @CircuitBreaker(name = "users", fallbackMethod = "fallbackPerson") 1 usage
17     public PersonResponseDTO getPersonByUsername(String username) {
18         String url = "http://micro-users/api/persons/by-username/" + username;
19         return restTemplate.getForObject(url, PersonResponseDTO.class);
20     }
21
22     @CircuitBreaker(name = "users", fallbackMethod = "fallbackUser") no usages
```

Eureka + API Gateway + Microservicios

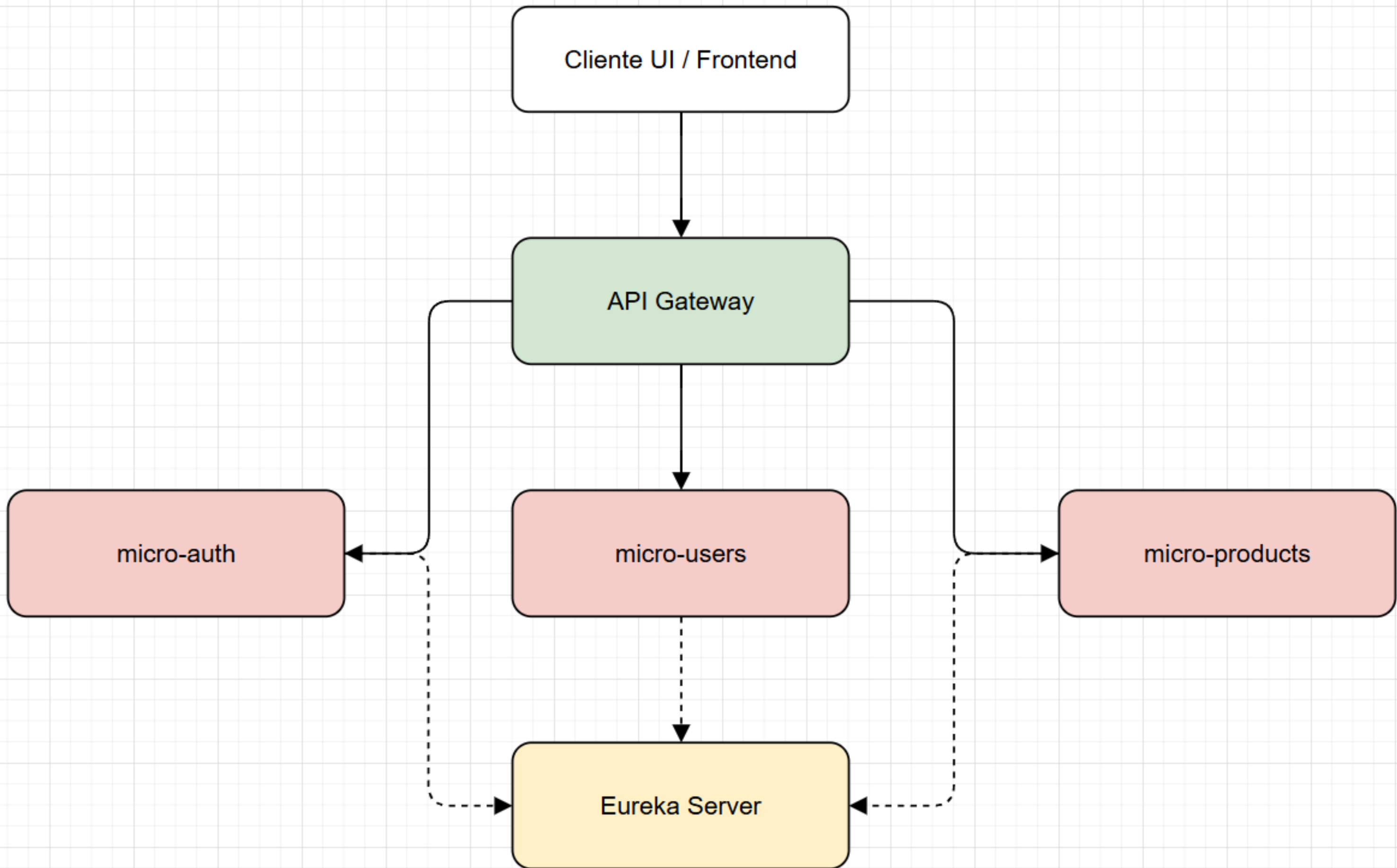
```
micro-products - \mvnw spring-boot:run -pl micro-products
2025-06-27T19:51:54.981-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Single vip registry refresh property : null
2025-06-27T19:51:54.984-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Force full registry fetch : false
2025-06-27T19:51:54.985-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Application is null : false
2025-06-27T19:51:54.987-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Registered Applications size is zero : true
2025-06-27T19:51:54.988-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Application version is -1: false
2025-06-27T19:51:54.989-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Getting all instance registry info from the eureka server
2025-06-27T19:51:55.003-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: The response status is 200
2025-06-27T19:52:00.016-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Disable delta property : false
2025-06-27T19:52:00.016-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Single vip registry refresh property : null
2025-06-27T19:52:00.017-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Force full registry fetch : false
2025-06-27T19:52:00.017-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Application is null : false
2025-06-27T19:52:00.018-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Registered Applications size is zero : true
2025-06-27T19:52:00.018-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Application version is -1: false
2025-06-27T19:52:00.018-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Getting all instance registry info from the eureka server
2025-06-27T19:52:00.044-05:00 INFO 30484 --- [micro-products] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: The response status is 200

api-gateway - \mvnw spring-boot:run -pl api-gateway
2025-06-27T19:51:55.639 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - Force full registry f
2025-06-27T19:51:55.640 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - Application is null :
2025-06-27T19:51:55.642 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - Registered Applicatio
2025-06-27T19:51:55.643 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - Application version i
s -1: false
2025-06-27T19:51:55.644 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - Getting all instance
registry info from the eureka server
2025-06-27T19:51:55.657 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - The response statu
s 200
2025-06-27T19:52:00.670 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - Disable delta prop
erty : false
2025-06-27T19:52:00.671 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - Single vip regist
ry refresh property : null
2025-06-27T19:52:00.674 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - Force full regist
ry fetch : false
2025-06-27T19:52:00.675 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - Application is n
ull : false
2025-06-27T19:52:00.676 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - Registered App
lications size is zero : true
2025-06-27T19:52:00.678 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - Application ve
rsion is -1: false
2025-06-27T19:52:00.679 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - Getting all i
nstance registry info from the eureka server
2025-06-27T19:52:00.697 [DiscoveryClient-CacheRefreshExecutor-%d] INFO c.netflix.discovery.DiscoveryClient - The response st
atus is 200

micro-auth - \mvnw spring-boot:run -pl micro-auth
Single vip registry refresh property : null
2025-06-27T19:51:55.409-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Force full registry fetch : false
2025-06-27T19:51:55.410-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Application is null : false
2025-06-27T19:51:55.410-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Registered Applications size is zero : true
2025-06-27T19:51:55.411-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Application version is -1: false
2025-06-27T19:51:55.411-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Getting all instance registry info from the eureka server
2025-06-27T19:51:55.418-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: The response status is 200
2025-06-27T19:52:00.421-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Disable delta property : false
2025-06-27T19:52:00.421-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Single vip registry refresh property : null
2025-06-27T19:52:00.422-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Force full registry fetch : false
2025-06-27T19:52:00.423-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Application is null : false
2025-06-27T19:52:00.423-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Registered Applications size is zero : true
2025-06-27T19:52:00.424-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Application version is -1: false
2025-06-27T19:52:00.424-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Getting all instance registry info from the eureka server
2025-06-27T19:52:00.458-05:00 INFO 25792 --- [micro-auth] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: The response status is 200

eureka-server - \mvnw spring-boot:run -pl eureka-server
: Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-06-27T19:51:39.782-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-1] o.s.web.servlet.DispatcherServlet
: Initializing Servlet 'dispatcherServlet'
2025-06-27T19:51:39.784-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-1] o.s.web.servlet.DispatcherServlet
: Completed initialization in 1 ms
2025-06-27T19:51:40.081-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-2] c.n.e.registry.AbstractInstanceRegistry
: Registered instance MICRO-PRODUCTS/host.docker.internal:micro-products:8092 with status UP (replication=false)
2025-06-27T19:51:40.392-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-4] c.n.e.registry.AbstractInstanceRegistry
: Registered instance MICRO-AUTH/host.docker.internal:micro-auth:8700 with status UP (replication=false)
2025-06-27T19:51:40.619-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-6] c.n.e.registry.AbstractInstanceRegistry
: Registered instance API-GATEWAY/host.docker.internal:api-gateway:8080 with status UP (replication=false)
2025-06-27T19:51:40.763-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-7] c.n.e.registry.AbstractInstanceRegistry
: Registered instance MICRO-PRODUCTS/host.docker.internal:micro-products:8092 with status UP (replication=true)
2025-06-27T19:51:40.763-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-7] c.n.e.registry.AbstractInstanceRegistry
: Registered instance MICRO-AUTH/host.docker.internal:micro-auth:8700 with status UP (replication=true)
2025-06-27T19:51:41.131-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-8] c.n.e.registry.AbstractInstanceRegistry
: Registered instance API-GATEWAY/host.docker.internal:api-gateway:8080 with status UP (replication=true)
2025-06-27T19:51:43.321-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-10] c.n.e.registry.AbstractInstanceRegistry
: Registered instance MICRO-USERS/host.docker.internal:micro-users:8091 with status UP (replication=false)
2025-06-27T19:51:43.824-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-1] c.n.e.registry.AbstractInstanceRegistry
: Registered instance MICRO-PRODUCTS/host.docker.internal:micro-products:8092 with status UP (replication=true)
2025-06-27T19:51:43.824-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-1] c.n.e.registry.AbstractInstanceRegistry
: Registered instance MICRO-AUTH/host.docker.internal:micro-auth:8700 with status UP (replication=true)
2025-06-27T19:51:43.824-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-1] c.n.e.registry.AbstractInstanceRegistry
: Registered instance API-GATEWAY/host.docker.internal:api-gateway:8080 with status UP (replication=true)
2025-06-27T19:51:43.824-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-1] c.n.e.registry.AbstractInstanceRegistry
: Registered instance MICRO-PRODUCTS/host.docker.internal:micro-products:8092 with status UP (replication=true)
2025-06-27T19:51:43.824-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-1] c.n.e.registry.AbstractInstanceRegistry
: Registered instance MICRO-AUTH/host.docker.internal:micro-auth:8700 with status UP (replication=true)
2025-06-27T19:51:43.824-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-1] c.n.e.registry.AbstractInstanceRegistry
: Registered instance API-GATEWAY/host.docker.internal:api-gateway:8080 with status UP (replication=true)
2025-06-27T19:51:43.824-05:00 INFO 23968 --- [eureka-server] [nio-8761-exec-1] c.n.e.registry.AbstractInstanceRegistry
: Registered instance MICRO-PRODUCTS/host.docker.internal:micro-products:8092 with status UP (replication=true)
2025-06-27T19:51:43.824-05:00 INFO 23968 --- [eureka-server] [nio-
```


API Gateway



Seguridad por ROL y PERMISO

© ProductController.java ×

```
37 public class ProductController {
38
39     private final ProductService productService;
40
41     @Operation(summary = "Crear un nuevo producto", description = "Crea un nuevo producto")
42     @ApiResponses(value = {
43         @ApiResponse(responseCode = "201", description = "Producto creado exitosamente",
44             content = @Content(mediaType = "application/json", schema = @Schema(implementation = ProductCreatedDTO.class))),
45         @ApiResponse(responseCode = "400", description = "Datos inválidos o entidades no encontradas")
46     })
47     @PostMapping
48     @PreAuthorize("hasAuthority('CREATE_PRODUCT')")
49     public ResponseEntity<?> createProduct(
50         @Parameter(description = "Datos del producto a crear", required = true)
51         @Valid @RequestBody ProductCreatedDTO productCreatedDTO) {
52         try {
53             ProductResponseDTO created = productService.create(productCreatedDTO);
54             return ResponseEntity.status(HttpStatus.CREATED).body(created);
55         } catch (IllegalArgumentException ex) {
56             return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(ex.getMessage());
57         }
58     }
59 }
```

Seguridad con JWT

POST http://localhost:8080/ap

GET http://localhost:8080/api/

GET http://localhost:8080/api/

+

...

HTTP

http://localhost:8080/api/auth/login

POST

http://localhost:8080/api/auth/login

Params

Authorization

Headers (8)

Body

none

form-data

x-www-form-urlencoded

1 {

2 "username": "admin",

3 "password": "1234"

4 }

Body

Cookies

Headers (4)

Test Results

Pretty

Raw

Preview

Visualize

JSON

1 {

2 "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbGlzInJvbGVzIjpbIkFTU0lHTl9ST0xFOX1BFUk1JU1NJT04iLCJDUkVBVEVfQVUUSE9SIiwiaWF0IjFQVRFX0NBVECIkNSRUFURV9QRVJTT04iLCJDUkVBVEVfUFJPRFVDVCIsIkNSRUFURV9ST0xFIiwiaWF0IjFQVRFX1NBTEUiLCJDUkVBVEVfU1RPQ0t9DSyIsIkRFTFVURV9BVVRIT1IiLCJERUxFeVUVEVfQ0FURUdPUlkiLCJERUxFeVUVEVfRURJVE9SSUFMIiwiaREVMRVFX1BFUlnPTiIsIkRFPTEUiLCJHRVRfQVUUSE9SIiwiaWV0VUX0FVVEhPUlMiLCJHRVRfQ0FURUdPUk1FuyIsIkdfVF9DQVRFR09SWSIsIkdfVF9FRElUT1JJQVF9QRVJNSVNTSU90IiwiaWV0VUX1BFUk1JU1NJT05TIiwiaWV0VUX1BFUlnPTiIsIkdfVF9QRVJTT05TIiwiaWV0VUX1BST0RVQ1QiLCJHRVF9TVE9DSyIsIkdfVF9ST0xFIiwiaWV0VUX1JPTEVTIiwiaWV0VUX1NBTEUiLCJHRVRfU0FMRVMiLCJHRVRfU1RPQ0tftU9WRU1FTlQiLCJWUX1VTRVJTIiwiaUkVDESR0VfU1RPQ0siLCJSRU1PVkVfUk9MRV9QRVJNSVNTSU90IiwiaUkVQT1JU19WSUVXIiwiaUk9MRV9BRE1JPUIiIsIlVQREFURV9DQVRFR09SWSIsIlVQREFURV9FRElUT1JJQUwiLCJWUERBVEVfUEVSU090IiwiaVVBEQVRFX1BST0RVQ1QiLCJWUERBVEVfUk9MRSIiLCJWQREFURV9TVE9DS19NT1ZFTUV0VCIsIlZJRvdG9uMX1BST0RVQ1RTIl0sImIhdCI6MTc1MTAwNDY1MiwiZXdtYyI4X2QCoUCT0TQJHT60uSAmCoBQdWtEh58kxTtTM",

3 "tokenType": "Bearer",

4 "person": {

5 "id": 15,

6 "firstName": "Carlos",

7 "lastName": "Pérez Gómez",

8 "dni": "12345678",

9 "gender": "MASCULINO",

10 "address": "Av. Siempre Viva 742",

Validacion de JWT en Microservicios

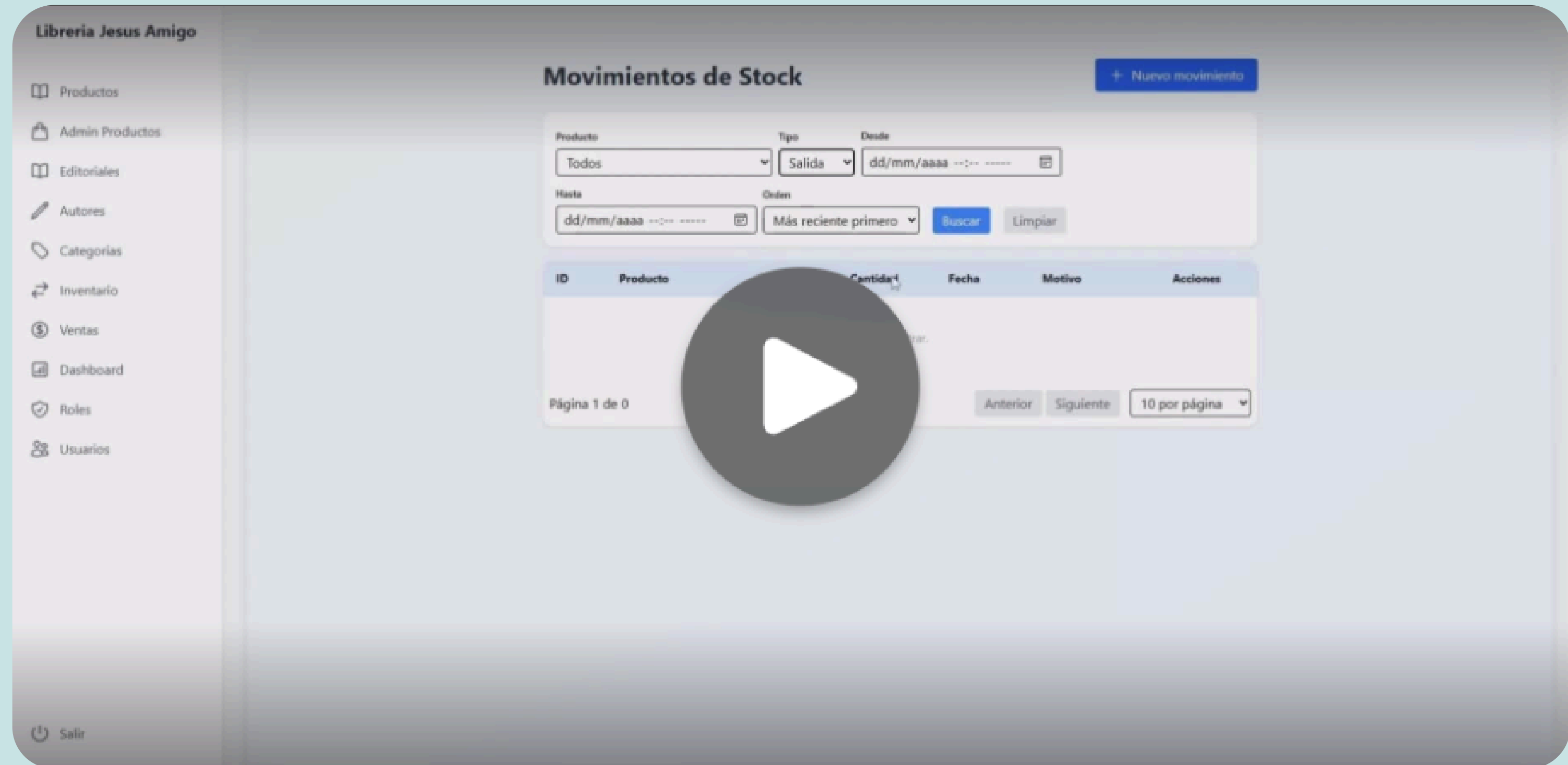
© JwtAuthFilter.java x

```
22      public class JwtAuthFilter implements GlobalFilter, Ordered {  
30  @      public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {  
31          String path = exchange.getRequest().getPath().toString();  
32          log.debug("Procesando petición a: {}", path);  
33  
34          // Permitir rutas públicas sin autenticación  
35          if (path.startsWith("/api/auth/")) {  
36              log.debug("Ruta pública - acceso permitido sin autenticación");  
37              return chain.filter(exchange);  
38          }  
39  
40          HttpHeaders headers = exchange.getRequest().getHeaders();  
41          log.debug("Cabeceras recibidas: {}", headers);  
42  
43          if (!headers.containsKey(AUTH_HEADER)) {  
44              log.warn("Intento de acceso sin cabecera Authorization");  
45              return unauthorized(exchange, reason: "Falta header Authorization");  
46          }  
47      }
```

Validacion de JWT en Microservicios

```
53
54     String token = authHeader.substring(TOKEN_PREFIX.length());
55     log.debug("Token JWT recibido: {}", token);
56
57     try {
58         log.debug("Validando token...");
59         if (!tokenProvider.validateToken(token)) {
60             log.warn("Token no válido: {}", token);
61             return unauthorized(exchange, reason: "Token inválido");
62         }
63
64         Claims claims = tokenProvider.parseClaims(token);
65         log.debug("Claims del token: {}", claims);
66
67         String username = claims.getSubject();
68         List<String> roles = claims.get("roles", List.class);
69
70         if (username == null || username.isEmpty()) {
```

Mostrar aplicacion



Gracias

